

Análisis y diseño con patrones

PR2: Patrones de diseño y de asignación de responsabilidades

Índice

Compromiso.....	2
Caso práctico.....	3
Pregunta 1 (25 %).....	5
Pregunta 1.1 (2,5 %).....	5
Pregunta 1.2 (10 %).....	7
Pregunta 1.3 (10 %).....	7
Pregunta 2 (30 %).....	11
Pregunta 2.1 (5 %).....	11
Pregunta 2.2 (15 %).....	13
Pregunta 2.3 (10 %).....	14
Pregunta 3 (30 %).....	17
Pregunta 3.1. (5 %).....	17
Pregunta 3.2 (15 %).....	19
Pregunta 3.3 (10 %).....	19
Pregunta 4 (15 %).....	21
Pregunta 4.1. (5 %).....	21

Compromiso

Certifico que he realizado la PRA2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Caso práctico

Se desea desarrollar una aplicación para la gestión de eventos sociales (como speed dating, intercambios de idiomas o networking), donde los usuarios pueden registrarse, participar en eventos y establecer conexiones con otros asistentes. La plataforma debe permitir gestionar usuarios, eventos, participaciones, preferencias y resultados de interacción entre participantes.

Las clases principales del sistema serían:

- **User:** Un usuario puede registrarse en múltiples eventos y expresar interés (*like*) por otros participantes. Cuenta con un identificador, un nombre, edad, email, un booleano para indicar si es usuario premium o no y una lista de intereses.
- **Event:** Representa un evento organizado en la plataforma. Un evento tiene una capacidad limitada y un conjunto de participantes registrados. Puede cambiar de estado según su evolución. Cuenta con un identificador, un título, una localización, una fecha, una capacidad o aforo máximo, un estado (*OPEN*, *FULL*, *CLOSED*) y el tipo de evento (*SPEEDDATING*, *AFTERWORK*, *NETWORKING*, *LANGUAGE_EXCHANGE* o *PREMIUM*).
- **Registration:** Permite modelar la relación entre usuarios y eventos, así como información adicional de la participación. Ese registro tendrá un identificador, una fecha de registro y un booleano para indicar si el usuario ha hecho check-in en ese evento (si ha aparecido).
- **Like:** Representa la preferencia de un usuario hacia otro dentro de un evento. Un usuario puede indicar interés por otros participantes del mismo evento. Cada *like* tendrá un identificador.
- **Match:** Representa una coincidencia entre dos usuarios. Un *match* se genera cuando existe interés mutuo entre dos usuarios. Cada *match* tendrá un identificador, así como una fecha.

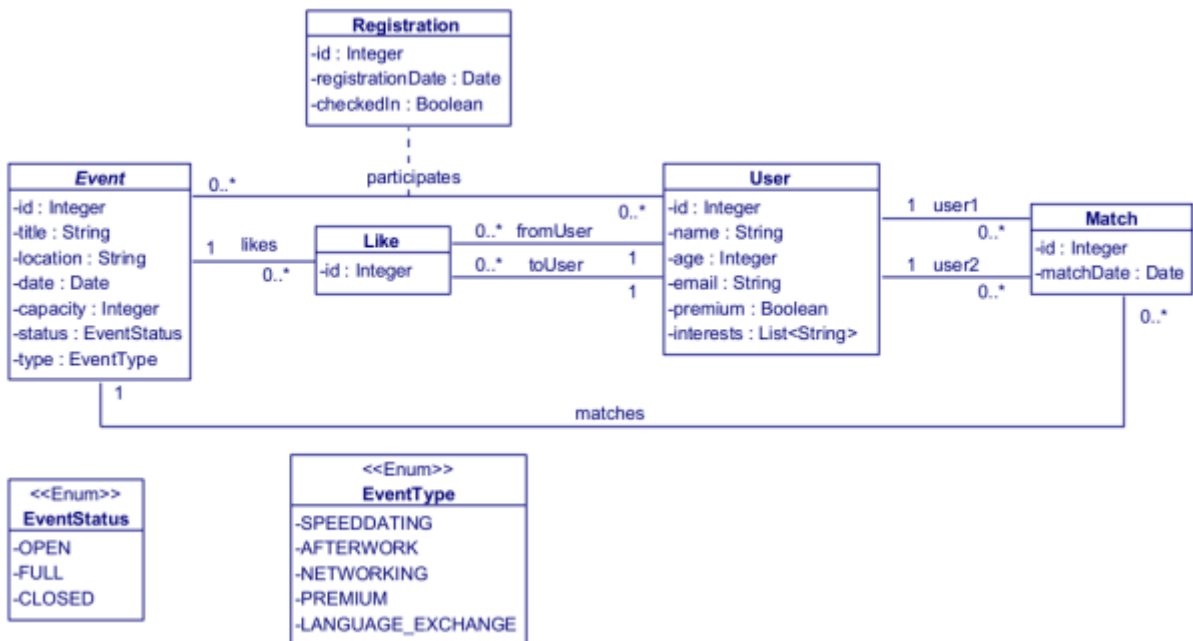
Un *User* puede participar en varios *Event*. Un *Event* contiene múltiples *User* (participantes). Un *User* puede dar múltiples *Like* dentro de un *Event* y finalmente un *Match* se genera a partir de dos *Like* recíprocos.

El sistema debe permitir:

- Gestionar eventos y participantes
- Registrar interacciones entre usuarios
- Generar resultados (*matches*)
- Notificar a los usuarios
- Ofrecer funcionalidades avanzadas (*premium*)

A partir de este contexto, se plantean distintas necesidades de evolución del sistema. El objetivo será **analizar cada situación y proponer soluciones de diseño adecuadas**, justificando el uso de patrones.

Aquí tenéis el diagrama UML de clases del sistema:



Restricciones de integridad:

- User.id, Event.id, Registration.id, Like.id y Match.id son únicos.
- Toda Registration debe estar asociada a exactamente 1 User y 1 Event.
- Todo Like debe estar asociado a 1 Event y a 1 fromUser y a 1 toUser donde fromUser y toUser no pueden ser el mismo usuario y deben ser del mismo Event.
- Todo Match debe estar asociado a exactamente 2 Users y a 1 Event. Los dos usuarios vinculados al match deben ser diferentes. Un match solo existe si hay reciprocidad y no puede haber duplicados: $\text{Match}(u1, u2) = \text{Match}(u2, u1)$.
- $\text{Event.participants.size} \leq \text{Event.capacity}$.
- Un User no puede registrarse 2 veces en el mismo Event

Pregunta 1 (25 %)

A partir del modelo inicial de la aplicación, formado por las clases *User*, *Event*, *Registration*, *Like* y *Match*, se desea incorporar una nueva funcionalidad para generar automáticamente los resultados de un evento una vez este haya finalizado.

Actualmente, un *Match* se crea cuando dos usuarios se han dado “like” mutuamente dentro del mismo *Event*. Sin embargo, la organización prevé que no todos los eventos utilicen siempre el mismo criterio para calcular los *matches*.

Por ejemplo, en algunos eventos se podrían aplicar las siguientes reglas:

- En eventos de tipo **speed dating**, solo se generará un match si existe reciprocidad directa entre dos participantes.
- En eventos **premium**, además de la reciprocidad, se tendrá en cuenta la edad, intereses comunes y prioridad de usuarios premium.
- En eventos de **networking**, se podrían generar sugerencias aunque no exista reciprocidad completa, siempre que haya un grado suficiente de afinidad.
- En eventos futuros, podrían incorporarse nuevos criterios de cálculo sin modificar el código existente de las entidades principales.

Además, el responsable técnico indica que no quiere que la clase *Event* ni la clase *Match* contengan estructuras condicionales complejas del tipo:

```
if (eventType.equals("SPEED_DATING")) {  
    ...  
} else if (eventType.equals("PREMIUM")) {  
    ...  
} else if (eventType.equals("NETWORKING")) {  
    ...  
}
```

Pregunta 1.1 (2,5 %)

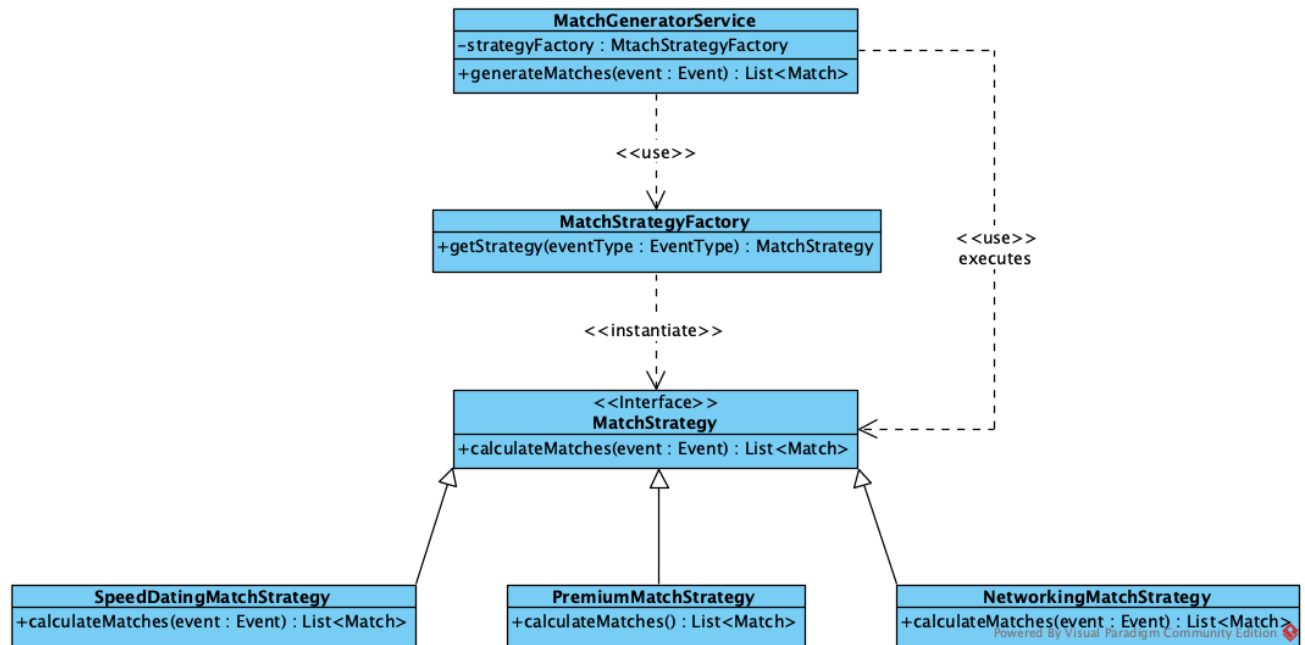
Para llevar a cabo la implementación de este caso de uso de forma eficiente y óptima, ¿qué patrón o patrones de diseño emplearías? Justifica la respuesta brevemente.

El enunciado indica explícitamente que existen diferentes formas de calcular los matches dependiendo del tipo de evento (*EventType*). Al aplicar el patrón **Estrategia**, crearíamos una interfaz común (por ejemplo, `MatchCalculationStrategy`) con un método como `calculateMatches(Event event)`. Luego, crearíamos clases concretas que implementen esta interfaz para cada *EventType*. De esta forma, eliminamos los bloques if-else en las clases principales y cumplimos con el Principio Abierto/Cerrado. Si en el futuro se añade un nuevo *EventType*, solo tenemos que programar una nueva clase estrategia sin tocar ni romper el código de *Event* o *Match*.

Por otro lado, también podemos apoyarnos del patrón **Factoría**. Aunque Estrategia resuelve cómo ejecutar el cálculo, seguimos necesitando un lugar donde decidir qué estrategia instanciar según el *EventType*. Podemos utilizar un método factoría que reciba el *EventType* y devuelva la instancia de la estrategia correspondiente. Así estaríamos centralizando la lógica de creación y manteniendo las clases de dominio (*Event*) completamente separadas de la lógica de instanciación de algoritmos.

Pregunta 1.2 (10 %)

Crea el diagrama de clases UML donde se vea claramente la aplicación del patrón o patrones empleados.



Pregunta 1.3 (10 %)

Escribe pseudocódigo que muestre cómo el sistema invoca el proceso de generación de matches de un evento desde el servicio asociado utilizando tu solución propuesta. No es necesario presentar el pseudocódigo de cómo se calculan al detalle exacto los matches en base a los likes del evento, sino que lo que interesa es que, mediante el pseudocódigo, se vea claramente la utilización del patrón o patrones usados para determinar qué regla utilizar. ¿Sería posible cambiar la forma de calcular los matches en tiempo de ejecución? ¿Cómo?

```
enum EventType {
    SPEEDDATING,
    PREMIUM,
    NETWORKING,
    AFTERWORK,
    LANGUAGE_EXCHANGE
}

class Event { ... }
class Match { ... }

interface MatchStrategy {
    List<Match> calculateMatches(Event event);
}

class SpeedDatingMatchStrategy implements MatchStrategy {
    @Override
    public List<Match> calculateMatches(Event event) {
        List<Match> matches = new ArrayList<>();

        // Lógica del algoritmo por desarrollar

        return matches;
    }
}

class PremiumMatchStrategy implements MatchStrategy {
    @Override
    public List<Match> calculateMatches(Event event) {
        List<Match> matches = new ArrayList<>();

        // Lógica del algoritmo por desarrollar

        return matches;
    }
}
```

```

class NetworkingMatchStrategy implements MatchStrategy {
    @Override
    public List<Match> calculateMatches(Event event) {
        List<Match> matches = new ArrayList<>();

        // Lógica del algoritmo por desarrollar

        return matches;
    }
}

class MatchStrategyFactory {
    public MatchStrategy getStrategy(EventType eventType) {
        return switch (eventType) {
            case SPEEDDATING -> new SpeedDatingMatchStrategy();
            case PREMIUM -> new PremiumMatchStrategy();
            case NETWORKING -> new NetworkingMatchStrategy();
            default -> throw new IllegalArgumentException("Tipo de
evento no soportado o sin estrategia asignada: " + eventType);
        };
    }
}

class MatchGeneratorService {
    // Relación de composición con la factoría
    private final MatchStrategyFactory strategyFactory;

    public MatchGeneratorService() {
        this.strategyFactory = new MatchStrategyFactory();
    }

    public List<Match> generateMatches(Event event) {
        if (event == null) {
            throw new IllegalArgumentException("El evento no puede ser
nulo");
        }

        // Extraemos el tipo de evento
        EventType type = event.getType();

        // La factoría decide qué estrategia instanciar de forma
centralizada
        MatchStrategy strategy = this.strategyFactory.getStrategy(type);

```

```

        // Ejecución polimórfica, ya que el servicio no sabe (ni le
        importa) qué reglas internas se aplican
        List<Match> calculatedMatches =
strategy.calculateMatches(event);

        return calculatedMatches;
    }
}

```

Respecto a la pregunta propuesta: sí que sería posible cambiar la forma de calcular los matches en tiempo de ejecución, ya que, al haber desacoplado el algoritmo del servicio principal, el comportamiento del sistema puede mutar dinámicamente en tiempo de ejecución sin necesidad de reiniciar la aplicación ni modificar el código. Existen dos formas principales de lograrlo según las necesidades del negocio:

1. Inyección o asignación dinámica

Podemos modificar el servicio o el método para permitir que un agente externo cambie la estrategia al vuelo pasándola como argumento:

```

// Permitir cambiar la estrategia directamente en el método si
fuera necesario
public generateMatchesCustom(Event event, MatchStrategy
estrategiaAlternativa): List<Match> {
    // Ignoramos la factoría y usamos la estrategia que nos
    inyectan en tiempo de ejecución
    return estrategiaAlternativa.calculateMatches(event);
}

```

2. Mutación del estado del propio *Event*

Dado que la factoría decide la estrategia basándose en el atributo *type* del objeto *Event*, si cambiamos el tipo de evento en tiempo de ejecución, el algoritmo cambiará automáticamente la próxima vez que se procese.

Por ejemplo, si un evento comienza siendo de tipo *SPEEDDATING* pero el organizador decide pulsar un botón para activar las funciones *PREMIUM* a mitad del evento, el sistema simplemente hace `event.setType(EventType.PREMIUM)`. Al invocar de nuevo al servicio, la factoría devolverá la estrategia premium (*PremiumMatchStrategy*).

Pregunta 2 (30 %)

Los arquitectos técnicos prevén que la plataforma evolucione y que se incorporen nuevos tipos de actividades. Además, consideran imprescindible que los diferentes tipos de eventos puedan tener reglas y datos específicos.

La organización quiere incorporar tres modalidades principales:

- **SpeedDatingEvent**: Representa un evento orientado a citas rápidas. Puede incluir información específica como el rango de edad, la duración del evento y el número máximo de matches que un usuario puede tener.
- **LanguageExchangeEvent**: Representa un evento de intercambio de idiomas. Puede incluir información específica como: idioma principal, idioma secundario y nivel.
- **NetworkingEvent**: Representa un evento profesional o de contactos. Puede incluir información específica como el sector, tema profesional y un booleano que indique si se requiere ser autónomo.

Actualmente, los eventos se crean desde diferentes partes del sistema: panel de administración, importación desde un formulario externo y herramienta interna de planificación. En cada una de estas partes se está empezando a repetir la lógica. El equipo técnico quiere evitar que esta lógica de creación quede repartida por toda la aplicación. Además, se prevé que en el futuro puedan aparecer nuevos tipos de evento, como: *BoardGameEvent*, *ConcertEvent* e *InvestmentTalkEvent* y no se desea modificar todas las pantallas o servicios que actualmente crean eventos.

Pregunta 2.1 (5 %)

Analiza el problema de diseño que aparece al tener distintos tipos de evento y múltiples puntos del sistema donde se crean objetos. Indica qué patrón de diseño aplicarías para encapsular y centralizar la creación de los diferentes tipos de evento. Justifica por qué este patrón es más adecuado que, por ejemplo, añadir condiciones if/else directamente.

El problema de diseño es que la lógica de instanciación de los distintos tipos de evento se está duplicando en varios puntos del sistema, como el panel de administración, el formulario externo de importación y la herramienta interna de planificación. Cada punto contiene condicionales para decidir qué clase concreta instanciar y cómo configurarla, lo cual genera código duplicado disperso en distintos módulos y alto acoplamiento a las clases concretas de evento, ya que cada creador debe conocerlas todas. Además, al añadir un nuevo tipo de evento, hay que modificar todos los lugares donde se crean eventos, lo que aumenta el riesgo de errores y rompe el principio de abierto/cerrado. Al no existir un punto único donde se apliquen reglas de validación o valores por defecto.

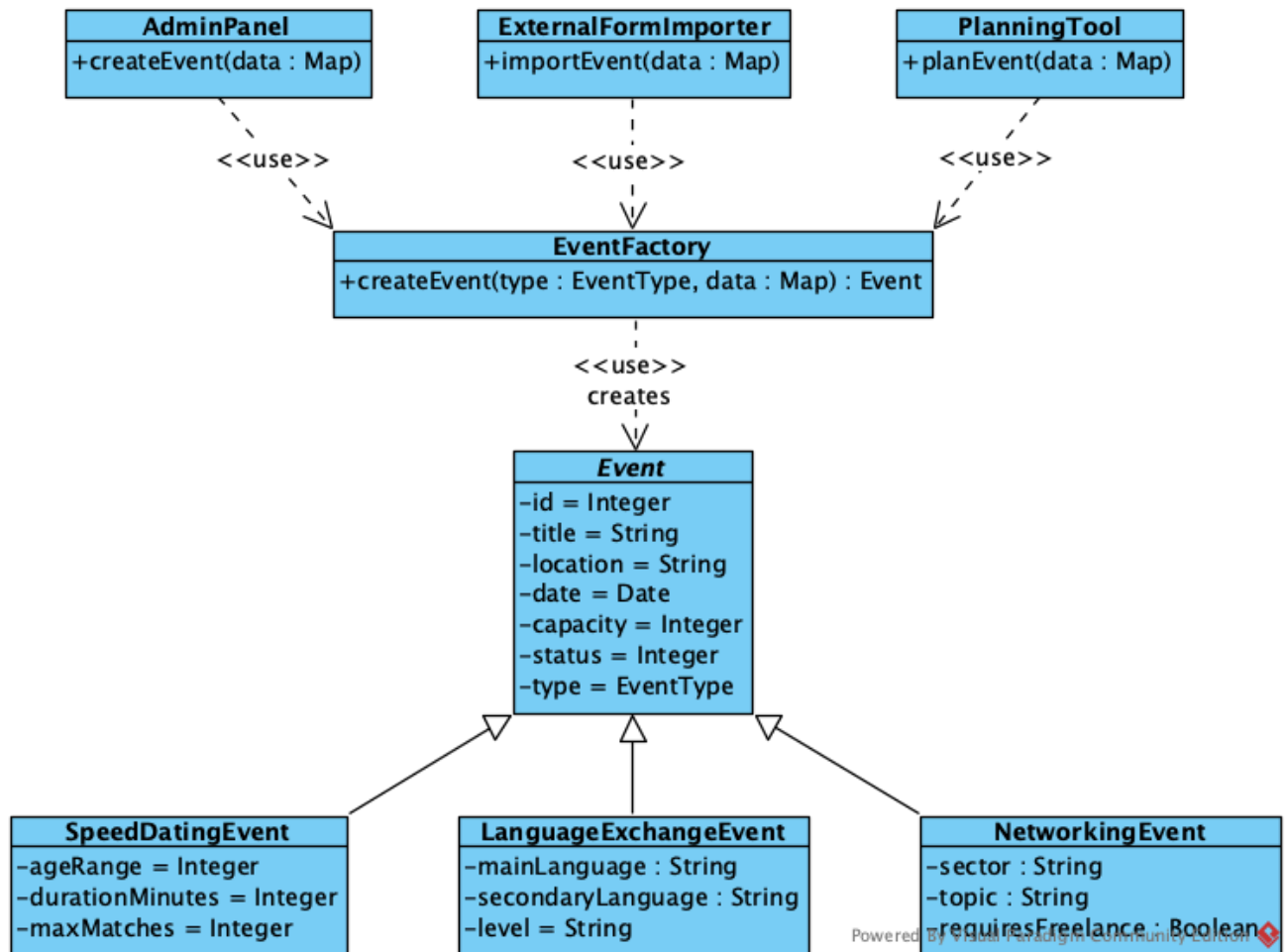
Para resolverlo, podemos aplicar el patrón **Método Factoría** (*Factory Method*). La idea es definir una interfaz o clase abstracta *EventFactory* con un método `createEvent(type, data)` que devuelva un *Event*. Las factorías concretas encapsulan la decisión de qué subclase instanciar y centralizan

toda la configuración específica. Los puntos del sistema que necesitan crear eventos simplemente invocan este método, sin conocer las subclases concretas.

Esta forma de atajarlo es más adecuado que un `if/else` porque elimina la duplicación, ya que: la lógica de selección de tipo y de construcción se escribe una sola vez en la factoría. Además, reducimos el acoplamiento, dado que los clientes dependen únicamente de la abstracción *Event* y de la factoría, no de cada subclase. Si queremos añadir un nuevo *EventType*, solo hay que modificar o extender la factoría, sin tocar las pantallas o servicios que ya crean eventos.

Pregunta 2.2 (15 %)

Propón un diagrama de clases UML teniendo en cuenta que por el momento tendremos tres tipos de eventos. No hay que dibujar todas las clases del sistema; solo las necesarias para entender el patrón o patrones de diseño aplicados.



Pregunta 2.3 (10 %)

Escribe el pseudocódigo simplificado de la parte del sistema mostrando cómo se invoca la creación del tipo de evento. No hay que implementar toda la lógica del caso de uso, solo la parte relacionada con la creación del evento según su tipo.

```
abstract class Event { ... }

class SpeedDatingEvent extends Event {
    private String ageRange;
    private int durationMinutes;
    private int maxMatches;

    // init, getters y setters específicos...
}

class LanguageExchangeEvent extends Event {
    private String mainLanguage;
    private String secondaryLanguage;
    private String level;

    // init, getters y setters específicos...
}

class NetworkingEvent extends Event {
    private String sector;
    private String topic;
    private boolean requiresFreelance;

    // init, getters y setters específicos...
}

// Factoría centralizada. Contiene la lógica condicional que antes se
// repetía en cada cliente.
class EventFactory {
    public Event createEvent(EventType type, Map<String, Object> data) {
        // Extraer campos comunes del mapa (simplificado)
        String id = (String) data.get("id");
        String title = (String) data.get("title");
        String location = (String) data.get("location");
        Date date = (Date) data.get("date");
        int capacity = (int) data.get("capacity");
        EventState state = (EventState) data.get("state");

        // Según el tipo, instanciar la clase correspondiente
    }
}
```

```

switch (type) {
    case SPEEDDATING:
        return new SpeedDatingEvent(
            id, title, location, date, capacity, state, type,
            (String) data.get("ageRange"),
            (int) data.get("durationMinutes"),
            (int) data.get("maxMatches")
        );
    case LANGUAGE_EXCHANGE:
        return new LanguageExchangeEvent(
            id, title, location, date, capacity, state, type,
            (String) data.get("mainLanguage"),
            (String) data.get("secondaryLanguage"),
            (String) data.get("level")
        );
    case NETWORKING:
        return new NetworkingEvent(
            id, title, location, date, capacity, state, type,
            (String) data.get("sector"),
            (String) data.get("topic"),
            (boolean) data.get("requiresFreelance")
        );
    case AFTERWORK:
    case PREMIUM:
    default:
        // Para estos tipos no hay subclase, se usa Event
        directamente
        return new Event(id, title, location, date, capacity,
            state, type);
}

// Ejemplo de cliente: panel de administración.
// Todos los clientes reciben los datos en un Map y obtienen el tipo de
// ahí.
class AdminPanel {

    private EventFactory eventFactory;

    public AdminPanel(EventFactory eventFactory) {
        this.eventFactory = eventFactory;
    }

    public Event createEvent(Map<String, Object> data) {

```

```
        // El tipo viene dentro de los datos (por ejemplo, un campo
        "eventType")
        EventType type = (EventType) data.get("eventType");
        // Se delega completamente en la factoría
        Event event = eventFactory.createEvent(type, data);
        // ... posteriormente se podrían realizar operaciones adicionales
        return event;
    }
}
```

Pregunta 3 (30 %)

Cada evento contiene un conjunto de participantes inscritos mediante objetos *Registration*. Hasta ahora, la aplicación únicamente necesitaba obtener la lista completa de asistentes para mostrarla en pantalla o generar los resultados del evento. Sin embargo, la organización quiere incorporar nuevas funcionalidades para mejorar la gestión interna de los eventos y ofrecer servicios adicionales a ciertos usuarios.

El equipo de producto ha detectado que los organizadores necesitan consultar los participantes de un evento de distintas formas sin exponer la estructura interna, según la situación. Hacen uso de la clase de gestión de eventos *EventManager* para obtener estos listados. Por ejemplo:

- Para preparar las mesas del speed dating, se necesita obtener los participantes en **orden de inscripción**.
- Para asignar turnos de conversación, se quiere obtenerlos en un **orden aleatorio**.
- Para revisar servicios especiales, se necesita filtrar únicamente por los usuarios que han contratado algún servicio adicional (es decir, son premium)

Paralelamente, la plataforma quiere ofrecer servicios opcionales a los participantes de un evento. Estos servicios pueden contratarse de forma independiente y combinarse entre sí. Algunos ejemplos son:

- Perfil destacado en la lista de participantes.
- Prioridad en la asignación de turnos.
- Mayor visibilidad en sugerencias de match.
- Acceso a estadísticas personales después del evento.

Un mismo usuario podría tener varios servicios al mismo tiempo. Se quiere que estos servicios puedan añadirse o retirarse sin modificar la clase User original, ya que esta clase forma parte del modelo base y se usa en muchas partes de la aplicación.

Pregunta 3.1. (5 %)

Justifica qué patrón o patrones de diseño se deben aplicar para implementar la casuística expuesta en el enunciado de manera que el acoplamiento se mantenga bajo y se cumpla el principio de Open-Closed.

Para cumplir con los dos requisitos podemos aplicar tres patrones:

1. **Strategy** para las consultas de participantes

Los organizadores necesitan consultar los participantes de un evento utilizando distintos criterios:

- Orden de inscripción.
- Orden aleatorio.
- Solo usuarios premium.

La variación principal se encuentra la forma de seleccionar u ordenar los participantes. Por ello, resulta adecuado aplicar el patrón Strategy. Para ello, definimos una interfaz común que represente un criterio de obtención de participantes y distintas estrategias concretas para cada necesidad.

De este modo, cada criterio se encapsula en una clase independiente y creamos la posibilidad de añadir nuevos criterios sin modificar las clases existentes. Así, *EventManager* únicamente conoce la abstracción de la estrategia.

2. **Iterador** para las consultas de participantes

Una vez obtenida la colección de participantes mediante una estrategia concreta, el acceso a los elementos los podemos realizar mediante el patrón Iterador, el cual proporciona una interfaz uniforme para recorrer la colección sin exponer su estructura interna.

Los clientes utilizan únicamente las operaciones:

- `hasNext()`
- `next()`

sin necesidad de conocer si los participantes proceden de una lista ordenada, filtrada o aleatorizada. Con esto, desacoplamos el mecanismo de recorrido de la representación interna de la colección.

3. **Decorator** para añadir servicios opcionales a los usuarios

Este patrón permite añadir responsabilidades a objetos de forma dinámica y transparente mediante envolturas. Para usarlo, definimos una interfaz o clase base común (por ejemplo, *Participant*) que *User* implementa.

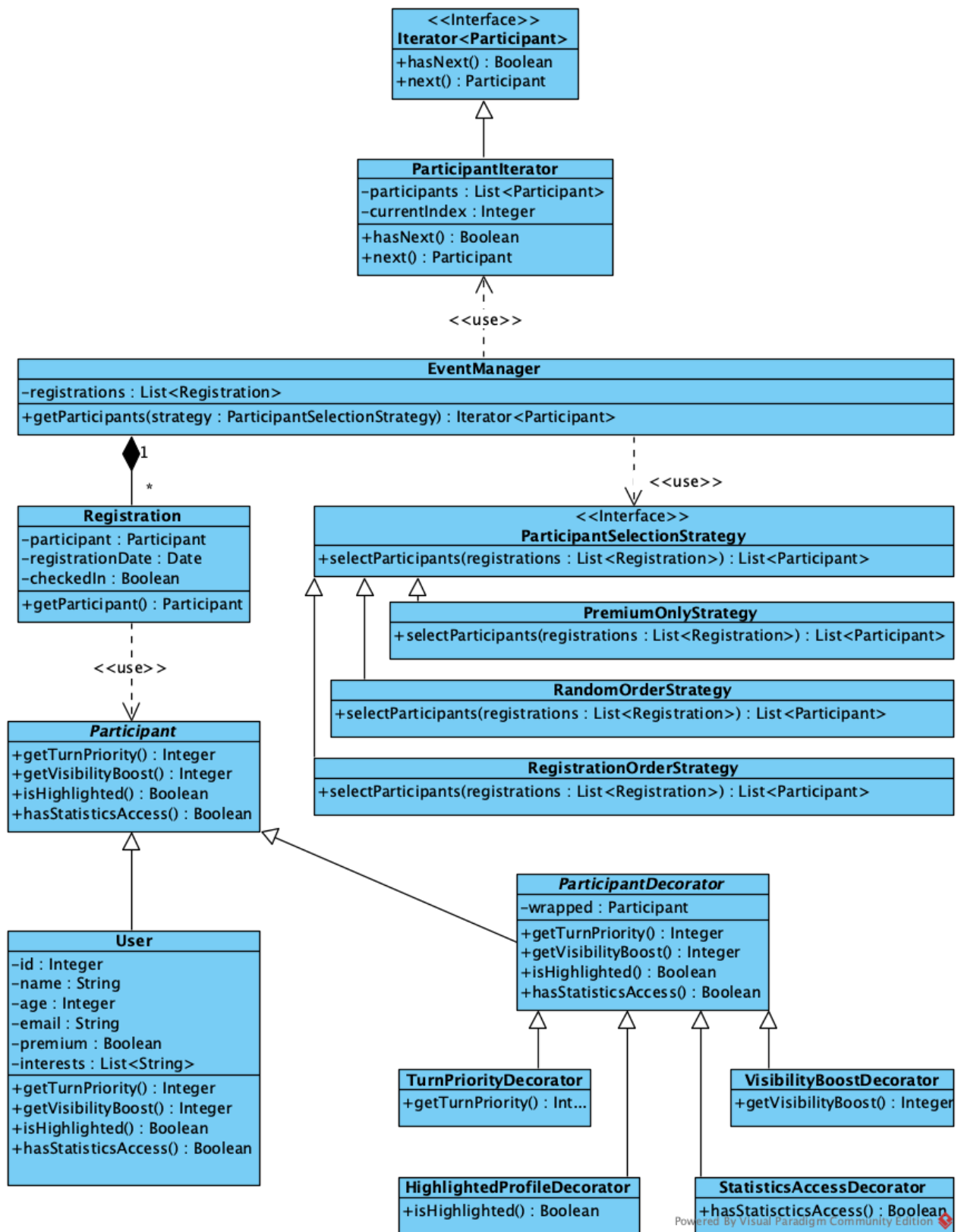
Cada servicio adicional se modela como un decorador que envuelve a otro *Participant* y añade la funcionalidad específica. Ejemplos: *HighlightedProfileDecorator*, *TurnPriorityDecorator*, *StatsDecorator*.

Podemos aplicar varios decoradores sobre un mismo *User*, y se pueden añadir o retirar en tiempo de ejecución.

Los nuevos servicios se incorporan creando nuevas clases decoradoras, sin tocar *User* ni los decoradores ya existentes, lo cual respeta el Principio Abierto/Cerrado. El acoplamiento se mantiene bajo debido a que el cliente interactúa con la abstracción *Participant* sin conocer su implementación concreta.

Pregunta 3.2 (15 %)

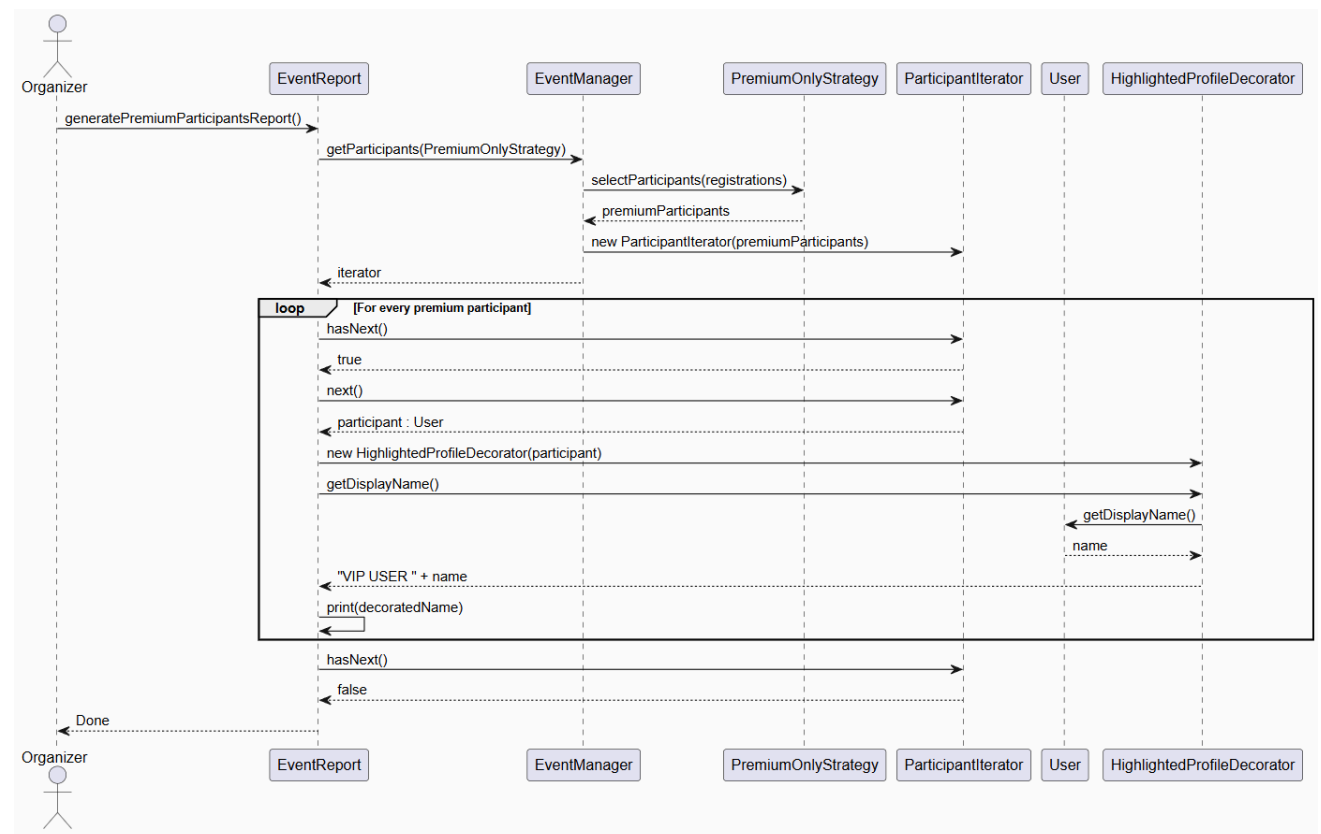
Muestra el diagrama de clases con las operaciones necesarias para implementar este diseño



Pregunta 3.3 (10 %)

Muestra el diagrama de secuencia del siguiente caso simplificado: Se quieren obtener los participantes “premium” y pintar todos los asistentes al evento en un informe. A medida que se van obteniendo, como por defecto estos cuentan con el servicio de perfil destacado, el sistema debe añadir dinámicamente delante del nombre de cada usuario el prefijo “VIP USER” que corresponde con el servicio opcional de perfil destacado.

Para este caso, crearemos un nuevo decorador llamado *HighlightedProfileDecorator*, el cual contará con un método llamado `getDisplayName()`.



Pregunta 4 (15 %)

La plataforma de eventos sociales ha ido incorporando nuevas funcionalidades alrededor de la gestión de eventos. Inicialmente, cerrar un evento consistía únicamente en cambiar el estado del Event a CLOSED. Sin embargo, con la evolución del sistema, el cierre de un evento implica ahora varias operaciones repartidas en distintos servicios.

Actualmente existen, entre otros, los siguientes componentes:

- **EventManager**: gestiona el estado del evento y sus inscripciones.
- **MatchingService**: calcula los Match a partir de los Like registrados.

Pero también tenemos estos otros componentes:

- **NotificationService**: envía mensajes a los participantes.
- **StatisticsService**: calcula métricas básicas del evento.

Cuando un organizador finaliza un evento, la aplicación debe ejecutar una secuencia completa de acciones:

1. Cambiar el estado del evento a CLOSED. Comprobar que el evento existe y está en estado válido para ser cerrado.
2. Generar los matches entre participantes y guardarlos.
3. Enviar notificaciones a los usuarios que han obtenido match.
4. Calcular estadísticas del evento.

Actualmente, esta lógica se ha implementado directamente en varios puntos del sistema. Cada cliente llama manualmente a los servicios anteriores y debe conocer el orden correcto de ejecución. Esto ha provocado multitud de problemas. Además, el equipo técnico prevé que en el futuro puedan añadirse nuevos pasos al cierre del evento, como publicar resultados en una app externa o enviar encuestas de satisfacción.

Pregunta 4.1. (5 %)

Para mejorar el diseño, ¿qué patrón o combinación de patrones sería más adecuado para optimizar el comportamiento planteado en el enunciado? Justifica tu respuesta.

El sistema actual delega en los clientes la responsabilidad de conocer e invocar múltiples servicios como *EventManager*, *MatchingService*, etc. en un orden estricto. Esto rompe el principio de encapsulamiento y genera un alto acoplamiento. Para evitarlos, podemos crear un nuevo componente, estructurado como una **Fachada** (por ejemplo, *EventClosureFacade*). Este componente expondrá una interfaz simplificada con un único método, como `closeEvent(eventId)`.

El cliente deja de interactuar con la complejidad de los distintos microservicios o componentes, abstrayendo así la complejidad. Por otro lado, centralizamos el flujo de control, puesto que la lógica de verificación de estado y el inicio del proceso de cierre quedan encapsulados en un solo

lugar. De esta forma prevenimos errores de consistencia provocados por llamadas en un orden incorrecto.

El enunciado establece como restricción previsible que "en el futuro puedan añadirse nuevos pasos al cierre del evento, como publicar resultados en una app externa o enviar encuestas". Integrar estas futuras dependencias directamente en la Fachada provocaría que esta creciera de forma descontrolada, violando el Principio de Responsabilidad Única y el Principio Abierto/Cerrado.

Para evitar esto, podemos usar el patrón de diseño **Observador**, ya que el cierre del evento debe modelarse como un evento de dominio (por ejemplo, *EventClosedNotification*). La Fachada, tras cambiar el estado a CLOSED, emite o publica esta notificación.

Al usar este patrón, los servicios secundarios (*MatchingService*, *NotificationService*, *StatisticsService*) actuarán como Observadores. Se mantendrán a la escucha y, al recibir la notificación, ejecutarán su lógica de forma independiente, desacoplando así su funcionalidad.

Asimismo, estaríamos aplicando correctamente el Principio Abierto/Cerrado porque, cuando se quiera añadir el envío de encuestas de satisfacción, bastará con crear un nuevo observador (por ejemplo, *SurveyService*) y suscribirlo al evento de cierre. El código de la Fachada y del sistema base permanecerá abierto a la extensión y cerrado a la modificación.

Pregunta 4.2. (10 %)

Crea un diagrama de clases UML donde se refleje la solución propuesta. Se tienen que mostrar claramente las clases principales en la aplicación del patrón o patrones.

